

Section 1

1. Random Number Generation

What values can this expression generate?

```
Random.nextInt(1, 7)
```

- This expression generates values 1, 2, 3, 4, 5, and 6. 7 is not included because the second parameter of nextInt sets the exclusive upper limit, so the starting value is included but the last value is not.

2. Main Thread

Why did this cause the application to stop responding normally?

```
Thread.sleep(100)
```

- It caused the application to stop responding normally because it blocks the thread for 100 milliseconds. If it is run on the main thread, it cannot update the UI while it's sleeping, making the app appear frozen or unresponsive.

3. ViewModel and Background Work

Does creating a ViewModel automatically cause all of its functions to execute on a background thread?

Explain

- No, creating a ViewModel does not automatically make its functions execute on background threads. It's mainly used to store and manage UI-related data. Its functions will normally execute on the thread that calls them. If background work is needed, we still need to use Threads or Coroutines. For example, a ViewModel can use viewModelScope.launch to start coroutine work.

4. Traditional Thread

Why did the Thread implementation require:

```
Handler(Looper.getMainLooper())
```

- Because the background thread should not directly update the user interface. Looper.getMainLooper() refers to the main UI thread, while the Handler allows code from the background thread to post work back to the main one.

5. Coroutine

What is the difference between:

```
Thread.sleep(100) and delay(100)
```

- Thread.sleep() blocks the entire thread, while delay() only suspends the coroutine. When using Thread.sleep(), the thread cannot perform other work until the set time has finished. In comparison, delay() temporarily pauses, but the thread can still be used for other tasks.

6. ViewModelScope

What is ViewModelScope?

Why is it useful when performing coroutine work from a ViewModel?

- viewModelScope is a CoroutineScope provided for a ViewModel. It allows us to launch coroutines directly inside the ViewModel. It is useful because the coroutine is connected to the lifecycle of the ViewModel. When it is cleared or destroyed, the coroutines running inside are automatically cancelled. This prevents unnecessary work from continuing when the ViewModel is no longer being used.

7. Thread vs. Coroutine

Compare the two implementations.

Which code requires more manual thread arrangement?

Which implementation is easier to read?

Explain your answer.

- The traditional Thread implementation requires more manual management. With Threads, we need to manually create them and then manually return to the main thread using the handler and looper. With Coroutines, the code is much simpler, and because of this, the Coroutine implementation is much easier to manage and read. Coroutines require less manual handling and work well with Android components such as ViewModel.

Section 2

1. Three of a Kind

For: 2 2 2 5 6

Is this Three of a Kind?

What is its score?

Explain how you calculated it.

- Yes, it is because there are 3 2s, which makes the condition true for isThreeOfAKind(). And the total score is calculated by getting the sum of all dice, which makes the score $2 + 2 + 2 + 5 + 6 = 17$.

2. Separation of Responsibilities

Explain the responsibility of each component.

YahtzeeScreen: What should it do?

- The YahtzeeScreen should be responsible for the UI. It displays the dice, buttons, scores, and categories. It should also be able to receive actions or events triggered by the player, such as pressing the Roll Button.

YahtzeeViewModel: What should it do?

- The YahtzeeViewModel should not be responsible for managing the game state and game flow. It should keep track of things such as:
 - Current dice values
 - Number of rolls
 - Selected or held dice
 - Current score
 - Selected scoring categories

The ViewModel acts as the connection between the UI and the rules

DiceRules: What should it do?

- DiceRules is responsible for the rules and scoring logic of the game. In our program, it contains functions such as:
 - isThreeOfAKind()
 - isFourOfAKind()
 - isFullHouse()
 - isSmallStraight()
 - isLargeStraight()
 - isYahtzee()

It also has the scoreFor() method, which calculates the score, and getAvailableCategories() that determines which scoring categories are currently valid.

3. Business Logic in Compose

Why would it be poor design to put code such as this directly inside YahtzeeScreen()?

```
if (dice.count { it == 6 } >= 3) {  
    // Three of a Kind  
}
```

Where should this logic be placed?

- It would be poor design to place the code directly in YahtzeeScreen() because it should mainly be responsible for displaying the UI and handling interaction. Placing game rules directly inside the Composable would mix the UI logic with the business logic, making it harder to maintain, reuse, and test. The logic can be placed inside DiceRules. The YahtzeeViewModel can call DiceRules when it needs to calculate or check a score, while the YahtzeeScreen displays the results.